

CTE412 · INTEGRATED SYSTEMS TECHNOLOGY

Docker Lab 2

Configuration, efficiency, and resilience

Unit IV · Advanced CI/CD Pipeline Practices

Module	CTE412 Integrated Systems Technology
Programme	BE in Information Technology
Unit	Unit IV — Advanced CI/CD Pipeline Practices
Topic	Bind mounts, settings, health checks, and Compose
Document type	Practical lab sheet (3 hours)
Prerequisites	Docker Step by Step must be completed first

How this lab works

This lab has two halves. The first six tasks are guided — every command is written out for you, with a check after each part so you know it worked. The final task is different: you are given only requirements, and you decide how to meet them. That final task is the one that is marked.

Everything here uses images you already downloaded in the previous session, so no internet is needed. Work through the guided tasks in order and keep to the times shown — you must reach the final task with at least 30 minutes left.

Contents

Before You Start	3
You will not download anything new	3
How this lab is organised	3
Prepare your folder	4
Final Task — Complete This on Your Own	17
The problem	17
Requirements	17
What to submit	17
How this is marked	18
Before you leave the lab	18
New Commands From This Lab	19

Before You Start

This lab continues directly from *Docker Step by Step*. You should already be able to run containers, read logs, build an image from a Dockerfile, use a volume, and start an application with Docker Compose. Today you will make those applications configurable, faster to build, and able to recover on their own.

You will not download anything new

Every task uses an image you already have from the previous session. If your images are still on your machine, this lab uses no internet at all.

Type this

```
$ docker images
```

You should see at least these

REPOSITORY	TAG	SIZE
nginx	alpine	52MB
python	3.12-alpine	52MB
redis	7-alpine	41MB
alpine	latest	7.8MB

CHECK YOUR WORK

If any of these four are missing, download them now before the lab gets busy: `docker pull nginx:alpine`, `docker pull python:3.12-alpine`, `docker pull redis:7-alpine`, and `docker pull alpine`.

How this lab is organised

There are **six guided tasks** followed by **one task you complete on your own**. The guided tasks give you every command. The final task gives you only requirements — you decide how to meet them, and it is the part that is marked.

Part	What you do	Time
Setting up	Check your tools and prepare a folder	10 min
Task 1	Edit a running site without rebuilding	20 min
Task 2	Configure a program without changing its code	25 min
Task 3	Make builds faster by controlling what you send	20 min
Task 4	Look inside an image and manage versions	20 min
Task 5	Make containers check themselves and recover	25 min
Task 6	Combine everything in one Compose file	30 min
Final task	Work on your own — this is marked	30 min

TIP

Keep an eye on the clock. If a guided task is taking much longer than the time shown, ask for help rather than falling behind — you must reach the final task with at least 30 minutes remaining.

Prepare your folder

Type this

```
$ mkdir docker-lab2
$ cd docker-lab2
```

Remove anything still running from last session so ports and names are free.

Type this

```
$ docker rm -f $(docker ps -aq)
```

NOTE

On Windows PowerShell that command is written differently: `docker rm -f (docker ps -aq)` without the dollar sign. If nothing is running you may see an error about no arguments — that is harmless.

Type this

```
$ docker ps -a
```

CHECK YOUR WORK

The list should be completely empty before you continue.

1

Editing Without Rebuilding — 20 minutes

In the last session, every time you changed a page you had to build the image again. That is slow while you are still working on something. A **bind mount** connects a folder on your computer straight into a running container, so your changes appear immediately.

1.1 Create a page

Make a folder and put a simple page in it.

Type this

```
$ mkdir site
$ cd site
```

Create this file with your editor:

```
FILE: INDEX.HTML
<!DOCTYPE html>
<html>
<body>
  <h1>Version one</h1>
</body>
</html>
```

1.2 Start a container connected to your folder

The command is different on each system because they write folder paths differently. Use the one that matches your computer.

Type this — macOS or Linux

```
$ docker run -d -p 8080:80 --name live \
-v "$(pwd)":/usr/share/nginx/html nginx:alpine
```

Type this — Windows PowerShell

```
$ docker run -d -p 8080:80 --name live -v ${PWD}:/usr/share/nginx/html nginx:alpine
```

The **-v** part means: *take the folder I am standing in, and make it appear inside the container at `/usr/share/nginx/html`.*

Open <http://localhost:8080> in your browser.

CHECK YOUR WORK

You should see **Version one**.

1.3 Change the page while it is running

In your editor, change **Version one** to **Version two**. Save the file. Refresh the browser.

CHECK YOUR WORK

The page now says **Version two**. You did not rebuild anything and you did not restart the container. The container is reading the file directly from your folder.

Change it once more to **Version three**, save, and refresh again. Each change appears straight away.

1.4 Understand the difference

Compare the two ways of getting a file into a container:

	COPY in a Dockerfile	Bind mount with -v
When the file arrives	When you build the image	Every time the container reads it
If you edit the file	Nothing changes until you rebuild	The change appears immediately
Where the file lives	Inside the image	On your computer
Works on another machine	Yes, the file travels with the image	No, that machine has no such folder
Use it for	Finished work you want to share	Work you are still changing

WARNING

A bind mount is for developing, not for delivering. If you gave this image to a classmate it would show the default Nginx page, because your folder does not exist on their machine. When your work is finished, use **COPY** in a Dockerfile so the file becomes part of the image.

1.5 Clean up

Type this

```
$ docker rm -f live
$ cd ..
```

2

Configuring a Program Without Changing Its Code — 25 minutes

Real programs need settings — a database address, a password, a language. You do not want to build a new image every time a setting changes. **Environment variables** let one image behave differently each time you run it.

2.1 Write a program that reads a setting

Type this

```
$ mkdir greeter
$ cd greeter
```

Create these three files:

FILE: APP.PY

```
import os
from flask import Flask

app = Flask(__name__)

# Read two settings. If they are missing, use these defaults.
GREETING = os.environ.get("GREETING", "Hello")
NAME = os.environ.get("NAME", "World")

@app.route("/")
def home():
    return f"<h1>{GREETING}, {NAME}!</h1>"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

FILE: REQUIREMENTS.TXT

```
flask==3.0.3
```

FILE: DOCKERFILE

```
FROM python:3.12-alpine
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

# Default settings, used if nobody supplies their own
ENV GREETING="Hello"
ENV NAME="World"

EXPOSE 5000
CMD ["python", "app.py"]
```

ENV sets a default inside the image. `os.environ.get` in the program reads it.

2.2 Build once

Type this

```
$ docker build -t greeter:1.0 .
```

2.3 Run the same image three different ways

First with no settings at all, so the defaults apply.

Type this

```
$ docker run -d -p 5000:5000 --name g1 greeter:1.0
```

Visit <http://localhost:5000>. It says **Hello, World!**

Now run a second container from the **same image**, supplying settings with `-e`.

Type this

```
$ docker run -d -p 5001:5000 --name g2 \
  -e GREETING="Kuzuzangpo" -e NAME="Bhutan" greeter:1.0
```

Visit <http://localhost:5001>.

CHECK YOUR WORK

The second page says **Kuzuzangpo, Bhutan!** while the first still says **Hello, World!** — two containers, two behaviours, one image, and no rebuilding.

Run a third with only one setting changed.

Type this

```
$ docker run -d -p 5002:5000 --name g3 -e NAME="CTE412" greeter:1.0
```

Visit <http://localhost:5002>.

CHECK YOUR WORK

It says **Hello, CTE412!** — the greeting kept its default because you did not replace it.

2.4 Look at the settings a container is using

Type this

```
$ docker exec g2 env
```

You will see a list that includes your two settings. This is useful when a container behaves unexpectedly — you can check what it was actually given.

2.5 Why this matters

The same image can now run as a test system and a live system, with different addresses and passwords, without one line of code changing. This is exactly how images move through a CI/CD pipeline: built once, then configured differently at each stage.

WARNING

Never write passwords or keys inside a Dockerfile using **ENV**. Anyone who has the image can read them. Passwords are supplied at run time with **-e**, or through a file that is never added to the image.

2.6 Clean up

Type this

```
$ docker rm -f g1 g2 g3
$ cd ..
```

3 Making Builds Faster — 20 minutes

When you build, Docker first collects every file in your folder and sends it to the background service. That collection is called the **build context**. If your folder contains large files you do not need, every single build wastes time sending them.

3.1 Go back to the greeter folder

Type this

```
$ cd greeter
$ docker build -t greeter:1.1 .
```

Look at the line in the output that mentions **load build context**. Note how long it takes and how many bytes it transfers. It should be very small.

3.2 Add something large

Programs often leave large files in their folder — installed libraries, caches, test data. Create one to see the effect.

Type this — macOS or Linux

```
$ dd if=/dev/zero of=bigfile.dat bs=1M count=200
```

Type this — Windows PowerShell

```
$ fsutil file createnew bigfile.dat 209715200
```

That makes a 200 MB file. Nothing in your Dockerfile mentions it. Build again.

Type this

```
$ docker build -t greeter:1.2 .
```

CHECK YOUR WORK

The **load build context** step now takes noticeably longer and reports around 200 MB. The file is never used, yet every build pays for it.

3.3 Exclude it

Create a file named exactly **.dockerignore** — note the dot at the beginning — in the same folder:

FILE: .DOCKERIGNORE

```
# Large files we never need in the image
bigfile.dat

# Things Python Leaves behind
__pycache__/
*.pyc
venv/

# Never put secrets or version history in an image
.env
.git
```

Type this

```
$ docker build -t greeter:1.3 .
```

CHECK YOUR WORK

The build context is small again and the build is fast. The 200 MB file is still sitting in your folder — it is simply no longer being sent.

3.4 Understand what .dockerignore really does

It works like `.gitignore`, but at a different moment. It decides what **leaves your computer**, before Docker even looks at your Dockerfile. So it helps even when no `COPY` line would have touched the file.

There are three reasons to always write one:

Reason	Why it matters
Speed	Nothing unnecessary is sent, so every build starts faster.
Size	Leftover files cannot accidentally end up inside your image.
Safety	A <code>.env</code> file or <code>.git</code> folder copied into an image can expose passwords to anyone who downloads it.

3.5 Clean up

Type this — macOS or Linux

```
$ rm bigfile.dat
```

Type this — Windows PowerShell

```
$ del bigfile.dat
```

Then

```
$ cd ..
```

4

Looking Inside an Image — 20 minutes

You have built several images now. This task shows you what is actually inside one, and how to manage versions properly.

4.1 See how an image was made

Type this

```
$ cd greeter
$ docker history greeter:1.3
```

You should see something like

IMAGE	CREATED	CREATED BY	SIZE
a1b2c3d4e5f6	2 minutes ago	CMD ["python" "app.py"]	0B
<missing>	2 minutes ago	EXPOSE map[5000/tcp:{}]	0B
<missing>	2 minutes ago	ENV NAME=World	0B
<missing>	2 minutes ago	ENV GREETING=Hello	0B
<missing>	2 minutes ago	COPY . . # buildkit	1.2kB
<missing>	3 minutes ago	RUN pip install ...	6.8MB
<missing>	3 minutes ago	COPY requirements.txt .	13B
<missing>	3 minutes ago	WORKDIR /app	0B
<missing>	2 weeks ago	/bin/sh -c #(nop) CMD ["pyth...	0B
<missing>	2 weeks ago	ADD file:... in /	52MB

Read it from the bottom up. The bottom lines came from `python:3.12-alpine`. The lines above are the instructions in **your** Dockerfile, one layer each.

CHECK YOUR WORK

Find the line for `RUN pip install`. It should be a few megabytes — that is Flask. Now find the lines for `ENV` and `EXPOSE`. They are **0B**, because they only record information; they add no files.

This explains why instruction order matters. Each line is a separate layer, and Docker reuses layers that have not changed.

4.2 Give an image more than one name

Your image is called `greeter:1.3`. A single image can carry several names at once.

Type this

```
$ docker tag greeter:1.3 greeter:latest
$ docker tag greeter:1.3 greeter:stable
$ docker images greeter
```

You should see

REPOSITORY	TAG	IMAGE ID	SIZE
greeter	1.3	a1b2c3d4e5f6	59MB
greeter	latest	a1b2c3d4e5f6	59MB
greeter	stable	a1b2c3d4e5f6	59MB

CHECK YOUR WORK

All three lines show the **same IMAGE ID**. There is only one image on your disk. The three names are labels pointing at it, so this uses no extra space.

4.3 Understand the word "latest"

Many people think `latest` means *the newest version*. It does not. It is simply the name Docker uses when you do not give one.

You proved this yourself just now: you attached `latest` to version 1.3 by hand. Nothing stops someone attaching it to an older version tomorrow.

WARNING

In real work, always ask for a specific version — `nginx:1.27-alpine`, not `nginx:latest`. If you build against `latest`, your project may work today and fail next week with no change to your code, which is extremely difficult to diagnose.

4.4 Remove a name without removing the image

Type this

```
$ docker rmi greeter:stable
$ docker images greeter
```

CHECK YOUR WORK

The message said **Untagged**, not **Deleted**. The **stable** name is gone but the image is still there, because other names still point to it. An image is only deleted when its last name is removed.

5

Containers That Recover on Their Own — 25 minutes

A container can be running but broken — the process is alive while the program inside has stopped responding. Docker cannot know that unless you tell it how to check. This task adds a **health check** and an automatic **restart**.

5.1 Add a health check

Stay in the `greeter` folder and edit your `Dockerfile`. Add these lines just before `CMD`:

FILE: DOCKERFILE

```
FROM python:3.12-alpine
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

ENV GREETING="Hello"
ENV NAME="World"

EXPOSE 5000

# Ask the program every 10 seconds whether it is still answering
HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
  CMD wget -q --spider http://localhost:5000/ || exit 1

CMD ["python", "app.py"]
```

Read the options:

Option	Meaning
<code>--interval=10s</code>	Run the check every 10 seconds.
<code>--timeout=3s</code>	If the check takes longer than 3 seconds, treat it as failed.
<code>--start-period=5s</code>	Allow 5 seconds for the program to start before judging it.

Option	Meaning
<code>--retries=3</code>	Only call it unhealthy after 3 failures in a row.
<code>wget -q --spider ...</code>	Ask the web page for a reply, without downloading it.

5.2 Watch the health status change

Type this

```
$ docker build -t greeter:2.0 .
$ docker run -d -p 5000:5000 --name checked greeter:2.0
$ docker ps
```

Look at the STATUS column. For the first few seconds it says **Up 3 seconds (health: starting)**.

Wait about fifteen seconds and run `docker ps` again.

CHECK YOUR WORK

The status should now read **Up 20 seconds (healthy)**. Docker is testing your program continuously and reporting the result.

Type this

```
$ docker inspect --format "{{.State.Health.Status}}" checked
```

This prints just the health word on its own, which is what an automated system would read.

5.3 Break it on purpose

Stop the program inside the container without stopping the container itself.

Type this

```
$ docker exec checked pkill -f "python app.py"
```

Wait about thirty seconds, then check.

Type this

```
$ docker ps -a
```

CHECK YOUR WORK

The container has stopped, or is reported as **unhealthy**. Without a health check, a broken program can sit there looking perfectly fine — which is why this matters.

Type this

```
$ docker rm -f checked
```

5.4 Make it restart automatically

A restart policy tells Docker what to do when a container stops unexpectedly.

Type this

```
$ docker run -d -p 5000:5000 --name resilient \
  --restart unless-stopped greeter:2.0
```

Now break it again.

Type this

```
$ docker exec resilient pkill -f "python app.py"

# wait about ten seconds, then
$ docker ps
```

CHECK YOUR WORK

The container is running again, and the STATUS column shows a small uptime such as **Up 8 seconds** — proof that it stopped and Docker started it again by itself.

Policy	What it does
no	Never restart. This is the default.
on-failure	Restart only if the program ended with an error.
always	Always restart, even after the computer is rebooted.
unless-stopped	Like always , but stays stopped if you stopped it deliberately.

TIP

unless-stopped is the sensible choice for most work. It recovers from crashes but respects your decision when you stop something on purpose.

Type this

```
$ docker rm -f resilient
$ cd ..
```

6**Putting It All Together in Compose — 30 minutes**

You now know four separate techniques. This task combines all of them into one file: settings, health checks, restart policies, and stored data.

6.1 Build the application**Type this**

```
$ mkdir visits
$ cd visits
```

Create these files:

```
FILE: APP.PY
import os
from flask import Flask
import redis

app = Flask(__name__)

SITE = os.environ.get("SITE_NAME", "My Site")
CACHE_HOST = os.environ.get("CACHE_HOST", "cache")
```

```

store = redis.Redis(host=CACHE_HOST, port=6379)

@app.route("/")
def home():
    hits = store.incr("hits")
    return f"<h1>{SITE}</h1><p>Visits: {hits}</p>"

@app.route("/health")
def health():
    store.ping()
    return "ok"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```

Notice the second address, `/health`. It does not just say the program is alive — it also checks that the database answers. A health check should test the things the program actually depends on.

FILE: REQUIREMENTS.TXT

```

flask==3.0.3
redis==5.0.8

```

FILE: DOCKERFILE

```

FROM python:3.12-alpine
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

EXPOSE 5000

HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
    CMD wget -q --spider http://localhost:5000/health || exit 1

CMD ["python", "app.py"]

```

FILE: .DOCKERIGNORE

```

__pycache__/
*.pyc
.env
.git

```

6.2 Write the Compose file

FILE: COMPOSE.YAML

```

services:
  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      SITE_NAME: "CTE412 Visit Counter"

```

```

    CACHE_HOST: "cache"
    restart: unless-stopped
    depends_on:
      cache:
        condition: service_healthy

cache:
  image: redis:7-alpine
  restart: unless-stopped
  volumes:
    - cachedata:/data
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s
    timeout: 3s
    retries: 3

volumes:
  cachedata:

```

6.3 Read what you just wrote

Line	What it does	From task
<code>environment:</code>	Supplies settings, the same job as <code>-e</code> .	Task 2
<code>restart: unless-stopped</code>	Brings a service back if it crashes.	Task 5
<code>healthcheck:</code>	Defines a check for a service that has none of its own.	Task 5
<code>condition: service_healthy</code>	Waits until the database is genuinely ready, not merely started.	Task 5
<code>volumes:</code>	Keeps the visit count when containers are removed.	Earlier
<code>.dockerignore</code>	Keeps the build small and safe.	Task 3

NOTE

The difference between `depends_on: - cache` and `condition: service_healthy` is important. The first only waits for the container to **start**. The second waits until the health check **passes**. A database container starts in a second but may take several more before it can answer — the first form lets your program connect too early and crash.

6.4 Run it

Type this

```
$ docker compose up -d --build
```

You should see

```

[+] Running 3/3
  Network visits_default      Created
  Container visits-cache-1    Healthy
  Container visits-web-1      Started

```

Notice the word **Healthy** beside the cache. Compose waited for it before starting the web service.

Type this

```
$ docker compose ps
```

CHECK YOUR WORK

Both services should be listed, and the STATUS column should show **(healthy)** for at least the cache. Visit <http://localhost:5000> and refresh several times — the visit count increases and the site name you set in the Compose file appears as the heading.

6.5 Prove that each piece works

The setting works. Stop everything, change `SITE_NAME` in `compose.yaml` to your own name, and start again:

Type this

```
$ docker compose down  
$ docker compose up -d
```

The heading changes, but the visit count carries on from where it was, because the volume kept it.

The restart policy works. Stop the program inside the web container:

Type this

```
$ docker compose exec web pkill -f "python app.py"  
  
# wait ten seconds, then  
$ docker compose ps
```

CHECK YOUR WORK

The web service is running again on its own. You have an application that survives a crash, waits for its database, keeps its data, and can be reconfigured without rebuilding.

6.6 Leave it running

Do not delete this application. You may look at it while you complete the final task.

Final Task — Complete This on Your Own

This is the part of the lab that is marked. From here on you receive **no commands and no examples**. Use what you have learned, your own notes, and `docker --help`. You may not copy files from the guided tasks.

WARNING

Read every requirement before you begin. Each one is checked separately when your work is marked.

The problem

A small library needs a system that records how many books have been borrowed. Build it using Docker.

Requirements

The application

1. A web service, written in Python, that runs in a container built from your own Dockerfile.
2. Visiting its main address increases a borrow count by one and displays the new total.
3. The library's name is displayed on the page, and is supplied as a setting when the service starts. It must **not** be written inside `app.py`.
4. A separate address reports whether the service and its storage are both working.

The storage

5. The count is stored in a separate container, not inside the web service.
6. The count survives when both containers are removed and started again.

Reliability

7. Both services restart automatically if they stop unexpectedly, but stay stopped if you stop them yourself.
8. Both services report a health status that Docker can display.
9. The web service does not start until the storage service is genuinely ready to answer.

Efficiency and safety

10. Your Dockerfile is ordered so that changing `app.py` does not reinstall the libraries.
11. Your build sends only the files that are needed.
12. Only images already on your machine are used. Nothing new is downloaded.

Starting it

13. The whole system starts with a single command.

What to submit

Upload one PDF to the VLE containing the following, in this order:

#	Item
1	A cover page with your name, student number, programme, and the date.
2	The complete contents of every file you wrote, each clearly titled.
3	A screenshot of the single command you used to start the system, and its output.
4	A screenshot showing both services running with their health status visible.
5	A screenshot of the page in your browser after at least three visits.
6	Evidence that the count survived removing and restarting the containers.
7	A short paragraph, no more than 150 words, explaining why you ordered your Dockerfile the way you did.

NOTE

Screenshots must be readable. Capture the terminal window only, not your whole screen, and do not shrink the image until the text blurs.

How this is marked

Marks	Criterion	What is looked for
8	Correctness, method, and process	Every requirement above is met and demonstrated by your evidence.
3	Readability of the solution	Files are tidy, sensibly named, and commented where a choice needs explaining.
2	Format and organisation	The submission follows the order listed above and is easy to follow.
2	Timely submission	Uploaded to the VLE before the deadline.

Before you leave the lab

Shut down anything still running and recover your disk space.

Type this

```
$ docker compose down
$ docker rm -f $(docker ps -aq)
$ docker system df
```

CHECK YOUR WORK

Your own work is saved in your folders, not in the containers. Removing containers does not delete your files.

WARNING

Do **not** run `docker volume prune` or `docker image prune -a` on a shared lab machine unless you are told to. You would delete the images your classmates are relying on, and they would have to download them all again.

New Commands From This Lab

Command	What it does
<code>docker run -v "\$(pwd)":/path ...</code>	Connect a folder on your computer into the container.
<code>docker run -e NAME=value ...</code>	Supply a setting to a container.
<code>docker exec <name> env</code>	Show the settings a running container is using.
<code>docker history <image></code>	Show the layers an image is built from.
<code>docker tag <image> <newname></code>	Give an image another name.
<code>docker inspect --format ... <name></code>	Print one specific detail about a container.
<code>docker run --restart unless-stopped ...</code>	Restart automatically after a crash.
<code>docker compose exec <service> <cmd></code>	Run a command inside a Compose service.

New Dockerfile and Compose entries

Entry	Where it goes	What it does
<code>ENV</code>	Dockerfile	Set a default setting inside the image.
<code>HEALTHCHECK</code>	Dockerfile	Tell Docker how to test that the program is working.
<code>.dockerignore</code>	Beside the Dockerfile	List files that must not be sent when building.
<code>environment:</code>	compose.yaml	Supply settings to a service.
<code>restart:</code>	compose.yaml	Choose what happens when a service stops.
<code>healthcheck:</code>	compose.yaml	Add a check to a service that has none.
<code>condition: service_healthy</code>	compose.yaml	Wait until another service is truly ready.